

Foundations of Cybersecurity / Applied Cryptography

26 July 2022

Exercise n. 1 [10]

State and prove the Shannon's Theorem.

Exercise n. 2 [12]

A client C and server S share a password P. Furthermore, C knows the public key Π_s of the server S. Client's and server's clocks are not synchronized.

Design a protocol that makes it possible to establish a symmetric session key k_{cs} between the client and the server. The protocol must avoid both the replay attack and the offline password guessing attack. Furthermore, it must satisfy the key authentication and key confirmation properties.

Exercise n. 3 [8]

Find and explain the vulnerabilities of function `EditStudent`, then patch them.

```
struct student{
int StudentID;
char FirstName[100];
char LastName[100];
int DegreeID;
}

void EditStudent(student* stu, int Sid, int Did, char* Fname, char* Lname) {
//The function replaces the value in the field of the structure pointed to by
// stu with the values Sid, Did, Fname, Lname
    stu->StudentID = Sid;
    strcpy(stu->FirstName, Fname);
    strcpy(stu->LastName, Lname);
    stu->DegreeID = Did;
}
```

Solution

EXERCISE N.1

See theory.

EXERCISE N.2

A possible protocol is the following:

M1 $S \rightarrow C: n_s$

M2 $C \rightarrow S: \mathcal{E}_{\Pi_s}(P, K_{cs}, n_s, n_c)$

M3 $S \rightarrow C: \mathcal{E}_{K_{cs}}(S, C)$

Notice that n_c is used to randomize message M2 and avoid an off-line password attack. Assume that session key K_{cs} gets compromised, and the adversary has recorded the protocol instance that installed that key. Without n_c , all the quantities in message M2 would be known except for the password P . Consequently, the adversary could use message M2 to mount an off-line brute force attack against the password P . Remember that Π_s is a public key.

For the same reason, quantity n_c must not be reported in M3. If n_c were in M3, then the adversary could retrieve by decrypting M3 with the compromised key. Thus, the adversary would be able to perform the off-line password attack described in the previous paragraph. Consequently, in order to guarantee key confirmation (aka direct authentication), it is necessary that the client newly generates the session key at each execution instance of the protocol (i.e., the session key is fresh).

EXERCISE N.3

The problem in this code is the function call `strcpy(dest, src)` that has an undefined behavior in case `src` point to a non-null-terminated character. “The behavior is undefined if either `dest` is not a pointer to a character array or `src` is not a pointer to a null-terminated byte string”.¹

The correct approach is to use `strncpy(dest, src, size)`² where parameter `size` specifies the size of the destination buffer minus one. Then, the programmer must put a null terminator into the last element of the destination buffer.

A possible new code is the following.

```
void EditStudent(student* stu, int Sid, int Did, char* Fname, char* Lname) {
//The function replaces the value in the field of the structure pointed to by
// stu with the values Sid, Did, Fname, Lname
    stu->StudentID = Sid;
    strncpy(stu->FirstName, Fname, 99);
    stu->FirstName[99]='\0';
    strncpy(stu->LastName, Lname, 99);
    stu->LastName[99]='\0';
    stu->DegreeID = Did;
    return;
}
```

¹ See <https://en.cppreference.com/w/c/string/byte/strcpy>

² See: <https://en.cppreference.com/w/c/string/byte/strncpy>